04 — Locks in PostgreSQL

Table of Contents

Learning Objectives

After studying this file you will be able to:

- Explain the four row-level lock modes and when each is acquired
- List all eight table-level lock modes and the SQL commands that acquire them
- Read the lock compatibility matrix and predict whether two operations will conflict
- Use advisory locks for application-level mutual exclusion
- Query `pg_locks` to observe active locks and identify blocking chains
- Write production-grade lock monitoring queries
- Identify the most dangerous lock anti-patterns and how to avoid them

---

Why Locking Exists Alongside MVCC

MVCC (covered in `02_mvcc.md`) solves the reader-writer blocking problem: plain `SELECT` never takes a row lock, so readers and writers do not block each other.

However, **writer-writer conflicts** cannot be resolved purely by version visibility. If two transactions both want to update the same row, one must wait for the other to commit or roll back — otherwise the second update would be based on a stale version.

PostgreSQL therefore uses two complementary mechanisms:

Mechanism	Solves	Acquired by
MVCC	Reader-writer non-blocking reads	Implicit (every query)
Locking	Writer-writer serialization	DML, DDL, explicit FOR UPDATE

Locks in PostgreSQL are held **until the end of the transaction** (not just the end of the statement), except for advisory locks which can be released explicitly.

Row-Level Locks

Row-level locks are acquired on individual tuples. They are stored **in the tuple header** (`xmax` field with special flags), not in a separate lock table (except for the queue of waiters). This means they scale to millions of rows without memory pressure.

The Four Row-Level Lock Modes

```
Strength (weakest to strongest):
  FOR KEY SHARE  <  FOR SHARE  <  FOR NO KEY UPDATE  <  FOR UPDATE
```

FOR KEY SHARE

The weakest row lock. Protects the key columns of the row from modification.

```
SELECT id, name FROM customers
WHERE id = 42
FOR KEY SHARE;
```

- Acquired by: referencing transactions in foreign key checks (the referenced row is locked FOR KEY SHARE so it cannot be deleted while the referencing row is being inserted/updated)
- Conflicts with: FOR UPDATE, FOR NO KEY UPDATE
- Does NOT conflict with: other FOR KEY SHARE, FOR SHARE

FOR SHARE

Protects the entire row from modification. Allows concurrent readers to share the lock.

```
SELECT * FROM orders
WHERE order_id = 100
FOR SHARE;
```

- Use case: "I am reading this row to make a decision; no one should change it while I decide."
- Conflicts with: FOR UPDATE, FOR NO KEY UPDATE
- Does NOT conflict with: FOR KEY SHARE, FOR SHARE

FOR NO KEY UPDATE

Locks the row for update but does NOT lock the key columns. Allows concurrent foreign key checks.

```
SELECT * FROM accounts
WHERE account_id = 7
FOR NO KEY UPDATE;
```

- Acquired by: `UPDATE` statements that do NOT modify any primary key or unique key columns
- Conflicts with: FOR SHARE, FOR KEY SHARE (actually, FOR NO KEY UPDATE conflicts with FOR SHARE but NOT with FOR KEY SHARE)
- Use case: updating non-key columns when you know foreign key relationships are unchanged

FOR UPDATE

The strongest row lock. Locks all columns including keys.

```
SELECT * FROM inventory
WHERE product_id = 55
FOR UPDATE;
```

- Acquired by: `UPDATE` and `DELETE` statements, and `SELECT ... FOR UPDATE`
- Conflicts with: ALL other row-level lock modes
- Use case: pessimistic locking pattern — "I will definitely update this row"

Row Lock Acquisition by DML

```
Statement          | Lock Acquired
-------------------+------------------------------
INSERT             | FOR UPDATE (on the new row)
UPDATE key columns | FOR UPDATE
UPDATE non-key cols| FOR NO KEY UPDATE
DELETE             | FOR UPDATE
FK check (ref row) | FOR KEY SHARE
```

NOWAIT and SKIP LOCKED

```
-- Fail immediately if row is locked (instead of waiting)
SELECT * FROM tasks
WHERE status = 'pending'
FOR UPDATE NOWAIT;

-- Skip any locked rows, return only unlocked ones (queue pattern)
SELECT * FROM tasks
WHERE status = 'pending'
ORDER BY created_at
LIMIT 10
FOR UPDATE SKIP LOCKED;
```

`SKIP LOCKED` is essential for building concurrent job queues. Multiple workers can each claim a batch of rows without blocking each other.

Locking Multiple Tables

```
-- Lock rows from two tables consistently
SELECT o.*, c.email
FROM orders o
JOIN customers c ON c.id = o.customer_id
WHERE o.id = 500
FOR UPDATE OF o          -- only lock rows in orders
FOR SHARE OF c;          -- lock customer rows more weakly
```

Table-Level Lock Modes

Table-level locks are acquired on the entire table. They are stored in shared memory (the lock manager). PostgreSQL has **eight table-level lock modes**, ranging from almost non-conflicting to fully exclusive.

All Eight Modes

```
Mode                | Abbreviation | Conflict Level
--------------------+--------------+---------------
ACCESS SHARE        | AS           | lightest
ROW SHARE           | RS           |
ROW EXCLUSIVE       | RE           |
SHARE UPDATE EXCL   | SUE          |
SHARE               | S            |
SHARE ROW EXCLUSIVE | SRE          |
EXCLUSIVE           | E            |
ACCESS EXCLUSIVE    | AE           | heaviest
```

ACCESS SHARE

- Acquired by: `SELECT` (plain read queries)
- Only conflicts with: ACCESS EXCLUSIVE
- Purpose: prevents the table from being dropped or restructured while it is being read

```
SELECT * FROM products;  -- acquires ACCESS SHARE on products
```

ROW SHARE

- Acquired by: `SELECT ... FOR UPDATE`, `SELECT ... FOR SHARE`
- Conflicts with: EXCLUSIVE, ACCESS EXCLUSIVE
- Purpose: signals that rows within the table are being locked for update

```
SELECT * FROM products WHERE id = 1 FOR UPDATE;  -- acquires ROW SHARE on products
```

ROW EXCLUSIVE

- Acquired by: `UPDATE`, `DELETE`, `INSERT`
- Conflicts with: SHARE, SHARE ROW EXCLUSIVE, EXCLUSIVE, ACCESS EXCLUSIVE
- Purpose: most DML runs under this mode

```
UPDATE products SET price = 99 WHERE id = 1;  -- acquires ROW EXCLUSIVE on products
```

SHARE UPDATE EXCLUSIVE

- Acquired by: `VACUUM` (non-FULL), `ANALYZE`, `CREATE INDEX CONCURRENTLY`, `ALTER TABLE ... VALIDATE CONSTRAINT`, `ALTER TABLE ... SET STATISTICS`
- Conflicts with: itself, SHARE, SHARE ROW EXCLUSIVE, EXCLUSIVE, ACCESS EXCLUSIVE
- Purpose: allows reads and DML to continue while maintenance runs, but prevents concurrent maintenance operations from conflicting with each other

```
VACUUM ANALYZE products;  -- acquires SHARE UPDATE EXCLUSIVE
```

SHARE

- Acquired by: `CREATE INDEX` (non-concurrent)
- Conflicts with: ROW EXCLUSIVE, SHARE UPDATE EXCLUSIVE, SHARE ROW EXCLUSIVE, EXCLUSIVE, ACCESS EXCLUSIVE
- Purpose: allows concurrent reads but blocks all writes while an index is built

```
CREATE INDEX idx_products_price ON products(price);  -- acquires SHARE
```

SHARE ROW EXCLUSIVE

- Acquired by: `CREATE TRIGGER`, `ALTER TABLE ... ADD FOREIGN KEY`
- Conflicts with: ROW EXCLUSIVE, SHARE UPDATE EXCLUSIVE, SHARE, SHARE ROW EXCLUSIVE, EXCLUSIVE, ACCESS EXCLUSIVE
- Purpose: similar to SHARE but also blocks concurrent SHARE locks

EXCLUSIVE

- Acquired by: `REFRESH MATERIALIZED VIEW CONCURRENTLY`
- Conflicts with: ROW SHARE, ROW EXCLUSIVE, SHARE UPDATE EXCLUSIVE, SHARE, SHARE ROW EXCLUSIVE, EXCLUSIVE, ACCESS EXCLUSIVE
- Allows: ACCESS SHARE (plain SELECTs still work)

ACCESS EXCLUSIVE

- Acquired by: `DROP TABLE`, `TRUNCATE`, `REINDEX`, `CLUSTER`, `VACUUM FULL`, `LOCK TABLE`, most `ALTER TABLE` forms
- Conflicts with: ALL other modes
- Purpose: completely exclusive access — blocks even plain SELECTs

```
ALTER TABLE products ADD COLUMN description text;  -- acquires ACCESS EXCLUSIVE
-- ALL concurrent queries on products must wait!
```

Lock Compatibility Matrix

A `Y` means the two modes can be held simultaneously. An `N` means they conflict — the second request must wait.

```
                    Existing Lock Held
Request          | AS  RS  RE  SUE  S  SRE  E   AE
-----------------+-----------------------------------
ACCESS SHARE     |  Y   Y   Y   Y   Y   Y   Y   N
ROW SHARE        |  Y   Y   Y   Y   Y   Y   N   N
ROW EXCLUSIVE    |  Y   Y   Y   Y   N   N   N   N
SHARE UPD EXCL   |  Y   Y   Y   N   N   N   N   N
SHARE            |  Y   Y   N   N   Y   N   N   N
SHARE ROW EXCL   |  Y   Y   N   N   N   N   N   N
EXCLUSIVE        |  Y   N   N   N   N   N   N   N
ACCESS EXCL      |  N   N   N   N   N   N   N   N
```

Practical Implications

```
Common Operation          | Lock Taken  | Blocked by
--------------------------+-------------+-----------------------------------------
Plain SELECT              | AS          | ACCESS EXCLUSIVE only
SELECT FOR UPDATE         | RS (table)  | EXCLUSIVE, ACCESS EXCLUSIVE
INSERT / UPDATE / DELETE  | RE          | SHARE, SHARE ROW EXCL, EXCL, ACCESS EXCL
CREATE INDEX (normal)     | S           | Any write DML
CREATE INDEX CONCURRENTLY | SUE         | Another SUE (e.g., VACUUM), not DML
ALTER TABLE (most forms)  | AE          | EVERYTHING — most dangerous
VACUUM (normal)           | SUE         | Another VACUUM/ANALYZE on same table
TRUNCATE                  | AE          | EVERYTHING
```

Advisory Locks

Advisory locks are **application-managed** locks with no automatic semantics. PostgreSQL just stores them; your application decides what they mean.

Why Advisory Locks?

- Row locks only exist for rows that exist — you cannot lock "the next row to be inserted"
- Table locks are too coarse
- Advisory locks let you implement arbitrary named mutexes inside the database

Session-Level Advisory Locks

Held for the entire session. Must be explicitly released (or session ends).

```
-- Acquire an exclusive advisory lock (blocks if already held)
SELECT pg_advisory_lock(12345);

-- Acquire a shared advisory lock
SELECT pg_advisory_lock_shared(12345);

-- Try to acquire (returns true/false immediately, never waits)
SELECT pg_try_advisory_lock(12345);          -- exclusive
SELECT pg_try_advisory_lock_shared(12345);   -- shared

-- Release
SELECT pg_advisory_unlock(12345);
SELECT pg_advisory_unlock_shared(12345);

-- Release all advisory locks held by this session
SELECT pg_advisory_unlock_all();
```

Transaction-Level Advisory Locks

Automatically released at COMMIT or ROLLBACK. Cannot be manually released early.

```
BEGIN;
  SELECT pg_advisory_xact_lock(42);           -- exclusive
```

```
  SELECT pg_advisory_xact_lock_shared(42);    -- shared

  -- Do protected work here
COMMIT;  -- lock released automatically
```

Two-Parameter Form (avoid collisions)

```
-- Use two int4 parameters to namespace locks by object type + id
-- Lock type=1 (table), id=9999 (some table OID)
SELECT pg_advisory_lock(1, 9999);

-- Lock type=2 (logical resource), id=42
SELECT pg_advisory_lock(2, 42);
```

Practical Advisory Lock Patterns

Pattern 1: Single-Instance Job Guard

```
-- Ensure only one instance of a job runs at a time
CREATE OR REPLACE FUNCTION run_nightly_report()
RETURNS void AS $$
DECLARE
  v_lock_acquired boolean;
BEGIN
  SELECT pg_try_advisory_lock(hashtext('nightly_report')) INTO v_lock_acquired;
  IF NOT v_lock_acquired THEN
    RAISE NOTICE 'Report already running, skipping.';
    RETURN;
  END IF;

  -- Do the work
  INSERT INTO report_results SELECT ...;

  PERFORM pg_advisory_unlock(hashtext('nightly_report'));
END;
$$ LANGUAGE plpgsql;
```

Pattern 2: Per-Account Serialization

```
-- Serialize operations per customer account without locking the whole table
BEGIN;
  -- Lock account 4567 specifically (no other session can process this account
simultaneously)
  SELECT pg_advisory_xact_lock(4567);

  -- Read-modify-write safely
  UPDATE accounts
  SET balance = balance - 100
  WHERE account_id = 4567
```

```
    AND balance >= 100;

  IF NOT FOUND THEN
    RAISE EXCEPTION 'Insufficient balance for account 4567';
  END IF;
COMMIT;
```

Pattern 3: Distributed Leader Election

```
-- Periodic check: am I the leader?
DO $$
DECLARE
  v_am_leader boolean;
BEGIN
  SELECT pg_try_advisory_lock(1) INTO v_am_leader;
  IF v_am_leader THEN
    RAISE NOTICE 'I am the leader, running work.';
    -- Do leader work; lock released when session ends
  ELSE
    RAISE NOTICE 'Another instance is the leader.';
  END IF;
END;
$$;
```

Observing Locks with pg_locks

`pg_locks` is a system view that shows all locks currently held or awaited across the entire cluster.

Key Columns

Column	Meaning
locktype	Type: relation, tuple, transactionid, advisory, object, etc.
relation	OID of the locked relation (for locktype=relation)
page / tuple	Physical location (for locktype=tuple)
transactionid	XID being waited on (for locktype=transactionid)
classid/objid	Two-integer key for advisory and object locks
virtualtransaction	Virtual transaction ID of the holder/waiter
pid	Backend process ID
mode	Lock mode (e.g., 'RowExclusiveLock', 'AccessShareLock')
granted	TRUE = lock is held; FALSE = lock is being waited for

Basic pg_locks Queries

```sql
-- Show all currently held and waited locks
SELECT pid, locktype, relation::regclass, mode, granted
FROM pg_locks
ORDER BY granted, pid;

-- Show only waiting locks (the blocked ones)
SELECT pid, locktype, relation::regclass, mode, granted
FROM pg_locks
WHERE NOT granted;

-- Show advisory locks
SELECT pid, classid, objid, mode, granted
FROM pg_locks
WHERE locktype = 'advisory';

-- Show row-level (tuple) locks
SELECT pid, relation::regclass, page, tuple, mode, granted
FROM pg_locks
WHERE locktype = 'tuple';
```

Lock Monitoring Queries

Query 1: Blocking Tree — Who Is Blocking Whom

```sql
-- Full blocking chain with query text
SELECT
    blocked.pid                         AS blocked_pid,
    blocked.usename                     AS blocked_user,
    blocked.application_name            AS blocked_app,
    blocked.query                       AS blocked_query,
    blocked.state                       AS blocked_state,
    now() - blocked.query_start         AS blocked_duration,
    blocker.pid                         AS blocker_pid,
    blocker.usename                     AS blocker_user,
    blocker.query                       AS blocker_query,
    blocker.state                       AS blocker_state,
    now() - blocker.xact_start          AS blocker_txn_duration
FROM pg_stat_activity blocked
JOIN pg_stat_activity blocker
    ON blocker.pid = ANY(pg_blocking_pids(blocked.pid))
WHERE cardinality(pg_blocking_pids(blocked.pid)) > 0
ORDER BY blocked_duration DESC;
```

Query 2: Lock Wait Summary by Table

```sql
-- Which tables are causing the most lock contention?
SELECT
    l.relation::regclass AS table_name,
```

```
    count(*) FILTER (WHERE NOT l.granted) AS waiting_locks,
    count(*) FILTER (WHERE l.granted)     AS held_locks,
    array_agg(DISTINCT l.mode)            AS modes
FROM pg_locks l
WHERE l.locktype = 'relation'
  AND l.relation IS NOT NULL
GROUP BY l.relation
ORDER BY waiting_locks DESC;
```

Query 3: All Lock Information in One View

```
-- Comprehensive lock view joining pg_locks with pg_stat_activity
SELECT
    l.pid,
    a.usename,
    a.application_name,
    l.locktype,
    CASE l.locktype
        WHEN 'relation'       THEN l.relation::regclass::text
        WHEN 'tuple'          THEN l.relation::regclass::text || ':' || l.page ||
':' || l.tuple
        WHEN 'transactionid'  THEN l.transactionid::text
        WHEN 'advisory'       THEN l.classid::text || ':' || l.objid::text
        ELSE '(other)'
    END AS lock_object,
    l.mode,
    l.granted,
    a.query,
    now() - a.xact_start AS txn_age
FROM pg_locks l
JOIN pg_stat_activity a ON a.pid = l.pid
ORDER BY l.granted, txn_age DESC NULLS LAST;
```

Query 4: Long-Held Locks

```
-- Locks held by transactions open more than 30 seconds
SELECT
    a.pid,
    a.usename,
    now() - a.xact_start AS txn_duration,
    l.locktype,
    l.relation::regclass AS table_name,
    l.mode,
    a.query
FROM pg_locks l
JOIN pg_stat_activity a ON a.pid = l.pid
WHERE l.granted
  AND a.xact_start IS NOT NULL
  AND now() - a.xact_start > interval '30 seconds'
```

```
  AND l.locktype = 'relation'
ORDER BY txn_duration DESC;
```

Query 5: Advisory Lock Inventory

```
-- All advisory locks with holder info
SELECT
    a.pid,
    a.usename,
    a.application_name,
    l.classid,
    l.objid,
    l.objsubid,    -- 1 = session-level, 2 = transaction-level
    l.mode,
    l.granted
FROM pg_locks l
JOIN pg_stat_activity a ON a.pid = l.pid
WHERE l.locktype = 'advisory'
ORDER BY l.classid, l.objid;
```

Query 6: Lock Queue Depth

```
-- How many waiters are behind each blocker?
SELECT
    blocker_pid,
    count(*) AS waiter_count,
    array_agg(blocked_pid ORDER BY wait_start) AS waiters
FROM (
    SELECT
        blocked.pid AS blocked_pid,
        unnest(pg_blocking_pids(blocked.pid)) AS blocker_pid,
        blocked.query_start AS wait_start
    FROM pg_stat_activity blocked
    WHERE cardinality(pg_blocking_pids(blocked.pid)) > 0
) t
GROUP BY blocker_pid
ORDER BY waiter_count DESC;
```

Lock Escalation and Lock Queuing

PostgreSQL does **not** escalate row locks to table locks. A transaction that locks 1 million rows holds 1 million row locks (in tuple headers) — but only one table-level ROW EXCLUSIVE lock.

Lock Queuing Behavior

When a lock cannot be granted immediately, the request enters a **wait queue** ordered by arrival time. This creates an important side effect: a queued ACCESS EXCLUSIVE request blocks all subsequent ACCESS SHARE requests even though ACCESS SHARE normally conflicts only with ACCESS EXCLUSIVE.

```
Timeline:
  T=0  Session A: SELECT (AS lock granted)
  T=1  Session B: SELECT (AS lock granted)
  T=2  Session C: ALTER TABLE (AE lock -- waiting, queued behind A and B)
  T=3  Session D: SELECT (AS -- waiting, queued behind C!)
                  Even though D is AS which doesn't conflict with A and B,
                  it must wait because C is ahead in the queue.
```

This is why a single `ALTER TABLE` during peak traffic can cause a pile-up of waiting SELECTs.

Mitigation: lock_timeout

```
-- Set a timeout so ALTER TABLE fails fast rather than queuing indefinitely
SET lock_timeout = '2s';
ALTER TABLE products ADD COLUMN description text;
-- If it cannot acquire the lock in 2s, raises: ERROR: canceling statement due to
lock timeout
```

DDL and Lock Interactions

```
DDL Statement                          | Lock Mode          | Blocks
---------------------------------------+--------------------+--------------------------
CREATE INDEX (non-concurrent)          | SHARE              | All writes
CREATE INDEX CONCURRENTLY              | SHARE UPDATE EXCL  | Other VACUUM/ANALYZE
DROP INDEX                             | ACCESS EXCLUSIVE   | Everything
ALTER TABLE ADD COLUMN (with default)  | ACCESS EXCLUSIVE   | Everything
ALTER TABLE ADD COLUMN (no default)    | ACCESS EXCLUSIVE   | Everything (but instant)
ALTER TABLE SET NOT NULL               | ACCESS EXCLUSIVE   | Everything
ALTER TABLE ADD CONSTRAINT (VALIDATE)  | SHARE UPDATE EXCL  | Validates without blocking
TRUNCATE                               | ACCESS EXCLUSIVE   | Everything
VACUUM FULL                            | ACCESS EXCLUSIVE   | Everything
VACUUM (normal)                        | SHARE UPDATE EXCL  | Other VACUUM
```

Safe Zero-Downtime DDL Pattern

```
-- Step 1: Add column with no default (fast, ACCESS EXCLUSIVE but instant)
ALTER TABLE orders ADD COLUMN processed_at timestamptz;

-- Step 2: Backfill in batches (no table lock)
UPDATE orders SET processed_at = created_at
WHERE processed_at IS NULL AND id BETWEEN 1 AND 100000;
-- ... repeat for other batches

-- Step 3: Add NOT NULL constraint using VALIDATE (no full table scan lock)
ALTER TABLE orders ADD CONSTRAINT orders_processed_at_nn
    CHECK (processed_at IS NOT NULL) NOT VALID;

ALTER TABLE orders VALIDATE CONSTRAINT orders_processed_at_nn;
```

```
-- VALIDATE uses SHARE UPDATE EXCLUSIVE — allows reads and writes

-- Step 4 (optional): Convert to actual NOT NULL (ACCESS EXCLUSIVE but fast once
validated)
ALTER TABLE orders ALTER COLUMN processed_at SET NOT NULL;
ALTER TABLE orders DROP CONSTRAINT orders_processed_at_nn;
```

Common Mistakes

1. SELECT FOR UPDATE Without Index

```
-- BAD: full table scan while holding row locks
SELECT * FROM orders WHERE status = 'pending' FOR UPDATE;
-- Holds locks on ALL scanned rows (even non-matching ones in some plans)

-- GOOD: ensure the WHERE clause is index-supported
CREATE INDEX idx_orders_status ON orders(status);
SELECT * FROM orders WHERE status = 'pending' FOR UPDATE;
```

2. Forgetting lock_timeout on DDL

```
-- BAD: this ALTER TABLE will queue and block all SELECTs indefinitely
ALTER TABLE products ADD COLUMN tags text[];

-- GOOD: fail fast, retry during a low-traffic window
SET lock_timeout = '3s';
ALTER TABLE products ADD COLUMN tags text[];
```

3. Releasing Advisory Locks on Transaction Abort

Session-level advisory locks are NOT released on ROLLBACK — only transaction-level ones are.

```
-- Session-level lock persists through ROLLBACK (potential surprise):
BEGIN;
  SELECT pg_advisory_lock(999);
ROLLBACK;
-- Lock 999 is STILL HELD by this session!
SELECT pg_advisory_unlock(999);  -- must release explicitly

-- Transaction-level lock is released automatically:
BEGIN;
  SELECT pg_advisory_xact_lock(999);
ROLLBACK;  -- lock 999 is released automatically
```

4. Assuming NOWAIT Is Free

`NOWAIT` raises an error immediately. This is appropriate for user-facing operations (fail fast), but using it in a retry loop without back-off can cause a thundering herd.

5. Long Transactions Holding Locks

```
-- BAD: opens a transaction, does expensive work, holds locks the whole time
BEGIN;
  SELECT * FROM accounts FOR UPDATE;    -- locks all account rows
  -- ... do expensive Python calculation for 10 seconds ...
  UPDATE accounts SET ...;
COMMIT;

-- GOOD: minimize lock-holding time
-- Do the expensive calculation BEFORE acquiring the lock
BEGIN;
  SELECT * FROM accounts WHERE id = $1 FOR UPDATE;  -- targeted lock
  UPDATE accounts SET ...;                           -- immediate update
COMMIT;
```

6. Deadlock via Lock Ordering

```
-- Session A:                          -- Session B:
BEGIN;                                 BEGIN;
UPDATE accounts SET bal=bal-10         UPDATE accounts SET bal=bal+10
  WHERE id=1;  -- locks row 1            WHERE id=2;  -- locks row 2
                                       UPDATE accounts SET bal=bal-10
UPDATE accounts SET bal=bal+10           WHERE id=1;  -- waits for A
  WHERE id=2;  -- waits for B
-- DEADLOCK: A waits for B, B waits for A
```

Best Practices

1. **Always specify `lock_timeout`** before DDL in production.
2. **Use `SELECT ... FOR UPDATE` only on rows you will actually modify** in the same transaction.
3. **Keep transactions short** — locks are held until COMMIT/ROLLBACK.
4. **Acquire locks in a consistent order** across all code paths to prevent deadlocks.
5. **Prefer transaction-level advisory locks** over session-level to avoid leaks.
6. **Monitor `pg_locks` in production** — alert when waiting lock count exceeds a threshold.
7. **Use `CREATE INDEX CONCURRENTLY`** instead of plain `CREATE INDEX` in production.
8. **Use `SKIP LOCKED` for job queues** instead of `FOR UPDATE` with serialized processing.
9. **Avoid `LOCK TABLE ... IN ACCESS EXCLUSIVE MODE`** in application code — it blocks everything.
10. **Set `idle_in_transaction_session_timeout`** to kill sessions that hold locks without making progress.

```
-- Recommended session defaults for production
SET lock_timeout = '5s';
SET idle_in_transaction_session_timeout = '2min';
SET statement_timeout = '60s';
```

Performance Considerations

Row Lock Storage

Row locks are stored in the tuple header (`xmax` + lock flags). For contested rows, PostgreSQL uses a **MultiXact** mechanism to store multiple row-lock holders in a single `xmax` slot. MultiXact IDs are allocated from `pg_multixact/` and must be vacuumed, similar to XID wraparound.

```
-- Check multixact age (should stay low)
SELECT datname, age(datminmxid) AS mxid_age
FROM pg_database
ORDER BY mxid_age DESC;
```

Lock Manager Overhead

The lock manager uses a hash table in shared memory. With many concurrent transactions each holding many locks, lock manager operations can become a bottleneck. Symptoms: high `lwlock:LockManager` wait events in `pg_stat_activity` .

```
-- Monitor lock manager waits
SELECT wait_event_type, wait_event, count(*)
FROM pg_stat_activity
WHERE wait_event_type = 'Lock'
GROUP BY wait_event_type, wait_event
ORDER BY count(*) DESC;
```

Index Coverage for Lock Efficiency

Unindexed `FOR UPDATE` clauses cause sequential scans with locks on all visited rows (not just the matching ones). Always ensure the filter predicate for a `FOR UPDATE` query is index-supported.

Connection Count and Lock Scalability

PostgreSQL's lock manager performs well up to a few hundred concurrent lock holders. Beyond that, contention on the `LockManager` lightweight lock increases. Use connection pooling (PgBouncer) to keep active connection counts manageable.

Interview Questions and Answers

Q1. What are the four row-level lock modes in PostgreSQL, and how do they differ?

A: From weakest to strongest: `FOR KEY SHARE` (protects key columns from deletion/key update), `FOR SHARE` (protects the whole row from any modification), `FOR NO KEY UPDATE` (acquired by UPDATE that does not touch key columns, blocks FOR SHARE but not FOR KEY SHARE), and `FOR UPDATE` (strongest, conflicts with all other row locks, acquired by UPDATE, DELETE, and explicit FOR UPDATE). The distinction between key and non-key locking enables foreign key checks to proceed concurrently with non-key UPDATEs.

Q2. Which table-level lock mode is acquired by a plain `SELECT` statement?

A: `ACCESS SHARE` . It is the weakest lock mode and only conflicts with `ACCESS EXCLUSIVE` (acquired by `DROP TABLE` , `TRUNCATE` , `ALTER TABLE` , etc.). This is why reads are never blocked by other reads or by

DML — they only conflict with schema changes.

Q3. What does `SELECT ... FOR UPDATE SKIP LOCKED` do and when would you use it?

A: `SKIP LOCKED` causes the `SELECT` to ignore (skip over) any rows that are currently locked by another transaction, rather than waiting for those locks to be released. It is used to implement concurrent work queues: multiple workers can each grab a batch of pending tasks without blocking each other.

Q4. What is an advisory lock and how does it differ from a regular row or table lock?

A: Advisory locks are application-defined named locks with no automatic database semantics. The application decides what the lock name means and what it protects. They come in session-level (persist until session ends or explicit release) and transaction-level (released at COMMIT/ROLLBACK) variants. Unlike row/table locks, they are not automatically acquired by SQL statements.

Q5. Why can a single `ALTER TABLE` statement cause a pile-up of waiting SELECT queries?

A: `ALTER TABLE` acquires `ACCESS EXCLUSIVE`, which conflicts with all lock modes including `ACCESS SHARE` (used by SELECT). But more importantly, because of PostgreSQL's lock queuing: once an `ACCESS EXCLUSIVE` request is queued (waiting for existing transactions to complete), all subsequent lock requests must queue behind it — even `ACCESS SHARE` requests that would normally be compatible with the currently held locks.

Q6. How do row locks interact with MVCC? Are they stored in pg_locks?

A: Row locks are stored in the tuple header's `xmax` field (with lock-mode flags), not in the central lock manager. This means they scale to millions of locked rows without shared memory pressure. They only appear in `pg_locks` as the waiter queue when a second transaction is blocked waiting for a row lock — not while they are uncontested.

Q7. What is the difference between `FOR UPDATE NOWAIT` and `FOR UPDATE SKIP LOCKED`?

A: `NOWAIT` raises an error immediately if the requested row is locked by another transaction ("ERROR: could not obtain lock on row in relation..."). `SKIP LOCKED` silently skips any locked rows and returns only the unlocked ones. Use `NOWAIT` when a conflict is unexpected and should be treated as an error. Use `SKIP LOCKED` when competing for a pool of work items.

Q8. What is `lock_timeout` and why is it critical for DDL in production?

A: `lock_timeout` is a GUC that causes a lock acquisition attempt to fail with an error if the lock cannot be granted within the specified interval. For DDL (which acquires ACCESS EXCLUSIVE), setting `lock_timeout` prevents a long-running `ALTER TABLE` from sitting in the lock queue indefinitely, blocking all subsequent reads and writes to that table. Without it, a single long transaction can trigger hundreds of blocked connections.

Q9. How do you determine which session is blocking another session?

A: Use `pg_blocking_pids(blocked_pid)` which returns the PIDs of all sessions that are blocking the given PID. Join with `pg_stat_activity` to get query text and start times. Alternatively, query `pg_locks` joining on relation/transactionid to find where `granted=false` rows share a lock object with `granted=true` rows.

Q10. What is a MultiXact and when does PostgreSQL use one?

A: When multiple transactions hold row-level locks on the same tuple simultaneously (e.g., multiple `FOR KEY SHARE` lockers), the `xmax` field can only hold one XID. PostgreSQL uses a `MultiXactId` — an opaque ID

that represents a set of XIDs — stored in `xmax`. The actual member XIDs are stored in `$PGDATA/pg_multixact/members`. MultiXacts must be vacuumed to prevent their own wraparound.

Q11. What happens to session-level advisory locks on ROLLBACK?

A: They are NOT released. Session-level advisory locks persist across transaction boundaries and survive ROLLBACK. Only transaction-level advisory locks (acquired with `pg_advisory_xact_lock`) are released at COMMIT or ROLLBACK. This is a common source of advisory lock leaks in applications that mix session-level advisory locks with error handling.

Q12. What lock does `CREATE INDEX` (non-concurrent) take and what does `CREATE INDEX CONCURRENTLY` take instead?

A: `CREATE INDEX` takes `SHARE` lock, which blocks all writes (`INSERT`, `UPDATE`, `DELETE`) for the duration of the index build. `CREATE INDEX CONCURRENTLY` takes `SHARE UPDATE EXCLUSIVE`, which allows reads and writes to proceed concurrently. The downside of CONCURRENTLY: it requires two scans of the table, takes longer overall, and can fail partway through (leaving an `INVALID` index that must be dropped and rebuilt).

Exercises and Solutions

Exercise 1 — Observe Row Lock Blocking

Open two psql sessions and observe blocking.

```
-- Session A:
BEGIN;
SELECT * FROM accounts WHERE id = 1 FOR UPDATE;
-- Do not commit yet

-- Session B:
SELECT * FROM accounts WHERE id = 1 FOR UPDATE;  -- blocks!

-- Session A: check what Session B is waiting for
SELECT blocked.pid, blocked.query, blocker.pid AS blocker_pid, blocker.query
FROM pg_stat_activity blocked
JOIN pg_stat_activity blocker ON blocker.pid = ANY(pg_blocking_pids(blocked.pid))
WHERE cardinality(pg_blocking_pids(blocked.pid)) > 0;

-- Session A:
COMMIT;
-- Session B immediately unblocks and acquires the lock.
```

Exercise 2 — Advisory Lock Guard

Write a PL/pgSQL function that uses an advisory lock to ensure it never runs concurrently with itself.

```
CREATE OR REPLACE FUNCTION safe_report_generation()
RETURNS text AS $$
DECLARE
  v_got_lock boolean;
BEGIN
```

```
  -- Try to acquire advisory lock (using a fixed application-defined ID)
  SELECT pg_try_advisory_xact_lock(20240101) INTO v_got_lock;

  IF NOT v_got_lock THEN
    RETURN 'Already running, skipped';
  END IF;

  -- Simulate work
  PERFORM pg_sleep(2);

  RETURN 'Report generated at ' || now()::text;
END;
$$ LANGUAGE plpgsql;

-- Call from two sessions simultaneously and observe only one runs
SELECT safe_report_generation();
```

Exercise 3 — Lock Compatibility Exploration

```
-- Step 1: Create test table
CREATE TABLE lock_test (id int PRIMARY KEY, val text);
INSERT INTO lock_test VALUES (1, 'a'), (2, 'b'), (3, 'c');

-- Step 2: Session A takes SHARE lock
BEGIN;
LOCK TABLE lock_test IN SHARE MODE;

-- Step 3: Session B tries ROW EXCLUSIVE (UPDATE) -- should block
UPDATE lock_test SET val = 'x' WHERE id = 1;

-- Step 4: In a third session, observe the conflict
SELECT l.pid, l.mode, l.granted, a.query
FROM pg_locks l
JOIN pg_stat_activity a ON a.pid = l.pid
WHERE l.relation = 'lock_test'::regclass;

-- Step 5: Session A commits, Session B proceeds
```

Exercise 4 — SKIP LOCKED Queue Worker

```
-- Setup: job queue table
CREATE TABLE job_queue (
    id         bigserial PRIMARY KEY,
    payload    jsonb,
    status     text DEFAULT 'pending',
    created_at timestamptz DEFAULT now()
);

INSERT INTO job_queue (payload)
```

```
SELECT json_build_object('task', i)
FROM generate_series(1, 20) i;

-- Worker function: claim up to 5 jobs atomically
CREATE OR REPLACE FUNCTION claim_jobs(p_batch_size int DEFAULT 5)
RETURNS SETOF job_queue AS $$
BEGIN
  RETURN QUERY
  UPDATE job_queue
  SET status = 'processing'
  WHERE id IN (
    SELECT id FROM job_queue
    WHERE status = 'pending'
    ORDER BY created_at
    LIMIT p_batch_size
    FOR UPDATE SKIP LOCKED
  )
  RETURNING *;
END;
$$ LANGUAGE plpgsql;

-- Run two workers simultaneously (they will claim different rows)
SELECT * FROM claim_jobs(5);  -- Session A
SELECT * FROM claim_jobs(5);  -- Session B (concurrently, gets different 5 rows)
```

Production Troubleshooting Scenarios

Scenario 1: "Waiting Lock" Pile-Up After Deploy

Symptom: After deploying a schema migration, hundreds of queries queue up and the application times out.

Diagnosis:

```
-- Find the blocking ALTER TABLE
SELECT pid, query, state, now() - xact_start AS age
FROM pg_stat_activity
WHERE query ILIKE '%ALTER TABLE%'
   OR state = 'idle in transaction'
ORDER BY age DESC;

-- Find what it is waiting for
SELECT * FROM pg_stat_activity
WHERE pid = ANY(pg_blocking_pids(<migration_pid>));
```

Fix:

1. Kill the long-running transaction that is preventing the ALTER TABLE from acquiring its lock.
2. Re-run the migration with `SET lock_timeout = '3s'` and retry logic.

Scenario 2: Advisory Lock Leak

Symptom: Application reports "could not acquire advisory lock" even though no application instance should be holding it.

Diagnosis:

```
-- Find who holds advisory lock 99999
SELECT a.pid, a.usename, a.application_name, a.state,
       now() - a.state_change AS idle_time
FROM pg_locks l
JOIN pg_stat_activity a ON a.pid = l.pid
WHERE l.locktype = 'advisory'
  AND l.objid = 99999;
```

Fix:

```
-- Terminate the leaking session (or it will release all its advisory locks)
SELECT pg_terminate_backend(<pid>);

-- Or release the specific lock from another connection if you know the session PID
-- (only advisory_unlock from the same session works)
```

Scenario 3: Row Lock Contention on Hot Row

Symptom: Many transactions queuing on a single row (e.g., a global counter or a "last updated" timestamp).

Diagnosis:

```
SELECT l.pid, l.granted, a.query, now() - a.xact_start AS age
FROM pg_locks l
JOIN pg_stat_activity a ON a.pid = l.pid
WHERE l.locktype = 'tuple'
ORDER BY l.granted DESC, age DESC;
```

Fix options:

- Use `pg_advisory_xact_lock(row_id)` and batch updates
- Redesign to avoid the hot row (sharding, append-only log + periodic aggregation)
- Use `SELECT counter FROM global_seq FOR UPDATE` only when truly needed

Cross-References

- `01_transaction_basics.md` — Transaction lifecycle; locks held until COMMIT/ROLLBACK
- `02_mvcc.md` — How MVCC and row locks coexist; xmax as both delete marker and lock token
- `03_isolation_levels.md` — How isolation levels interact with locking
- `05_deadlocks.md` — What happens when two transactions wait on each other's locks
- `06_optimistic_vs_pessimistic.md` — Choosing between SELECT FOR UPDATE and version-based approaches
- `07_serializable_snapshot_isolation.md` — SSI uses predicate locks, not traditional row locks
- `../10_PostgreSQL_Internals/03_lock_manager.md` — Internal lock manager hash table and LWLocks